

# AWS Task Manager

[ <http://65.0.11.141> ]

## Project Overview:

The AWS Task Manager is a simple full-stack web application developed to manage day-to-day tasks. I have developed this project to get hands-on experience with full-stack web development and AWS deployment.

The application allows a user to log in and manage tasks. Users can add, view, edit, complete and delete tasks. The frontend is built with React.js, while the backend provides REST APIs using Node.js and Express.js. PostgreSQL is used for storing the task information.

The application is hosted on an AWS EC2 instance, with Nginx used to serve the frontend and forward API requests to the backend.

I have also integrated AWS S3 and SNS to gain practical experience with AWS services. The main purpose of this project was to understand how a complete web application works from frontend development to backend APIs, database connectivity, and deployment on AWS.

## Project Objective:

The main objectives of this project were:

- To build a working application using React.js and Node.js.
- To create REST APIs for task management.
- To connect the backend with a PostgreSQL database.
- To deploy the application on AWS.
- To understand how services such as EC2, S3 and SNS can be used in an application.
- To gain practical experience with hosting and managing a full-stack application.

## Features:

- User login
- Add tasks
- View tasks
- Edit tasks
- Mark tasks as completed
- Delete tasks
- PostgreSQL database
- REST APIs

- React-based frontend
- AWS-hosted application
- S3 integration
- SNS integration

## Technology Stack:

### ➤ Frontend

- React.js
- JavaScript
- CSS
- React Router

### ➤ Backend

- Node.js
- Express.js

### ➤ Database

- PostgreSQL

### ➤ AWS / Deployment

- Amazon EC2
- Amazon S3
- Amazon SNS
- Nginx
- PM2

## Architecture:

The basic application flow is:

**User**



**AWS EC2**



**Nginx**



**React Frontend**



**Node.js + Express.js**



**PostgreSQL**

**Additional AWS Services: S3, SNS**

**Process Management: PM2**

**Access Management: IAM**

## Working:

1. When a user adds a task, the React frontend sends a request to the backend.
2. The request is handled by Nginx and forwarded to the Node.js/Express server.
3. The backend then stores the task in PostgreSQL and sends the result back to the frontend.
4. The frontend receives the response and updates the task list displayed to the user.

## REST API:

| Method | Endpoint                   | Description              |
|--------|----------------------------|--------------------------|
| GET    | "/api/tasks"               | Get all tasks            |
| POST   | "/api/tasks"               | Add a task               |
| PUT    | "/api/tasks/:id"           | Update a task            |
| PUT    | "/api/tasks/:id/completed" | Update completion status |
| DELETE | "/api/tasks/:id"           | Delete a task            |
| GET    | "/api/s3-test"             | Test S3 integration      |

## Database:

The project uses PostgreSQL.

The main task table contains:

id  
task  
completed  
created\_at

The backend uses parameterized queries when communicating with PostgreSQL.

## AWS Deployment:

The application is deployed on an AWS EC2 instance.

The deployment includes:

- **EC2** — Hosts the application
- **Nginx** — Serves the React frontend and acts as a reverse proxy
- **PM2** — Keeps the Node.js backend running
- **S3** — Provides cloud object storage integration

- SNS — Provides notification/messaging integration

The live application can be accessed here:

<http://65.0.11.141>

## Project Structure:

```
Task Manager/
├── backend/
│   ├── server.js
│   ├── package.json
│   └── .env
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   └── ...
└── README.md
```

«The ".env" file contains configuration values and should not be uploaded to GitHub or shared publicly. »

## Running the Project Locally (Commands):

### Backend

```
cd backend
npm install
node server.js
```

### Frontend

```
cd frontend
npm install
npm run dev
```

The frontend can then be opened using the local URL shown by Vite.  
(Ex: <http://localhost:5173>)

## **Future Improvements:**

Some improvements I would consider for a production version are:

- JWT authentication
- HTTPS and a custom domain
- Docker
- Application Load Balancer
- Redis for caching
- OpenSearch for advanced search
- CloudWatch for monitoring
- ECS/Fargate for containerized deployment

## **Conclusion:**

This project helped me understand the complete flow of a full-stack application, from building the React frontend and REST APIs to connecting PostgreSQL and deploying the application on AWS.

It gave me practical experience with AWS EC2, S3, SNS, Nginx and PM2.